Project Title:

Sudoku Solver Agent using CSP and Backtracking Search

Course Name & Number:

Artificial Intelligence CCE414

Date Due:

5/10/2026

Date Handed In:

5/10/2026

Team Members:

#	Student Name	Student ID	Section
1	Ramy Said Eliwa Elsayed	231903795	لائحة قديمة
2	David George Helmy Ekladious	231903654	لائحة قديمة
3	Ahmed Mohamed Abdelrazik	232903959	لائحة قديمة

Abstract

This report details the development of an AI agent designed to solve Sudoku by modeling it as a Constraint Satisfaction Problem (CSP). By implementing a Backtracking Search algorithm enhanced with the Minimum Remaining Values (MRV) heuristic, we achieved significantly faster solution times, often resolving puzzles in less than a second. Furthermore, the report explores Local Beam Search within the "Free Topic" section, discussing its alternative application and efficiency in addressing Sudoku challenges.

Acknowledgments

We would like to express our sincere appreciation to ***Dr. Rihanni*** for teaching and guiding us throughout this course. His lectures and explanations provided us with the fundamental knowledge and understanding needed to complete this project.

We are grateful for the effort and time dedicated to delivering the course material and helping us build a stronger background in the subject. The knowledge gained during this course played an important role in the development of this report.

Finally, we would also like to thank everyone who contributed, directly or indirectly, to the completion of this work.

Table of Contents:

1) Problem Formulation as a Search Problem

Introduction:

The goal of this project is to **create an AI agent that solves Sudoku.** We formulated Sudoku as a Constraint Satisfaction Problem (CSP). The aim is to fill a 9x9 grid with numbers from 1 to 9. The rules are simple: no repeating numbers in any row, column, or 3x3 block.

Rules (Constraints) :

1. **Do not repeat the number in the same row**
2. **Do not repeat the number in the same column**
3. **Do not repeat the number in the same 3×3 Block.**

Search Problem Formulation:

- **State:** A 9x9 grid with some starting numbers and empty spots (shown as 0).
- **Initial State:** The unsolved Sudoku board we start with.
- **Actions (Successor Function):** Putting a valid number (1-9) into an empty spot without breaking any Sudoku rules.
- **Goal Test:** Checking if all 81 spots are filled correctly and all rules (Row, Column, Box) are followed.
- **Path Cost:** Path cost doesn't matter here because any valid, finished board is a good solution.

2) Tools and Technologies Used

We used these tools for our project:

- **Programming Language: Python 3.** Python is great because its syntax is clear and it handles grids (like the Sudoku board) very easily.
- **Cloud Computing & Prototyping (Google Colab):** For our early development, we relied on Google Colab. This cloud-based tool let our team work together on the code in real-time, making it easy to test and fix bugs without having to set up anything on our own computers.
- **Development Environment (IDE):** We wrote and tested the code using **Visual Studio Code (VS Code)** / **PyCharm**.

3) Technical Discussion

Our engineering approach to solving the Sudoku puzzle relies on treating it as a **Constraint Satisfaction Problem (CSP)**. Instead of using a naive brute-force method to generate all possible grid combinations, the agent intelligently navigates the state space using a recursive **Depth-First Search (DFS)** algorithm, enhanced by **Backtracking** and an optimization heuristic.

Below is the step-by-step logical execution of our algorithm:

Step 1: Intelligent Cell Selection (The MRV Heuristic)

The algorithm begins by searching for an empty cell (represented by `0`). To minimize the search space, the agent uses the **Minimum Remaining Values (MRV)** heuristic via the `get_mrv_cell` function.

- **Scanning:** It scans all 81 cells and calculates the number of legal options (1-9) available for each empty cell based on the current board state.
- **Algorithmic Optimizations:**
    - *Immediate Backtrack:* If it finds a cell with `0` options, it instantly stops and returns that cell, forcing the program to backtrack immediately **(saving massive processing time)**.
    - *Immediate Assignment:* If it finds a cell with exactly `1` option, it selects it instantly without checking the rest of the board.
- **Default Action:** If neither extreme is found, it simply selects the cell with the fewest available options. If no empty cells are left, the puzzle is solved.

Step 2: Constraint Validation

Once an optimal empty cell is selected, the agent attempts to place a number in it (testing sequentially from 1 to 9). Before making the placement, the number is passed through the `is_valid` function.

- **The Rule Enforcer:** This function acts as the constraint checker. It verifies the three fundamental Sudoku rules:
    1. Does the number exist in the current **Row**?
    2. Does the number exist in the current **Column**?
    3. Does the number exist in the local **3x3 Sub-grid**?
- If the number violates any of these constraints, the function rejects it, and the agent moves on to test the next number.

Step 3: State Assignment and Recursion

If the `is_valid` function approves the number, the agent temporarily assigns that number to the empty cell.

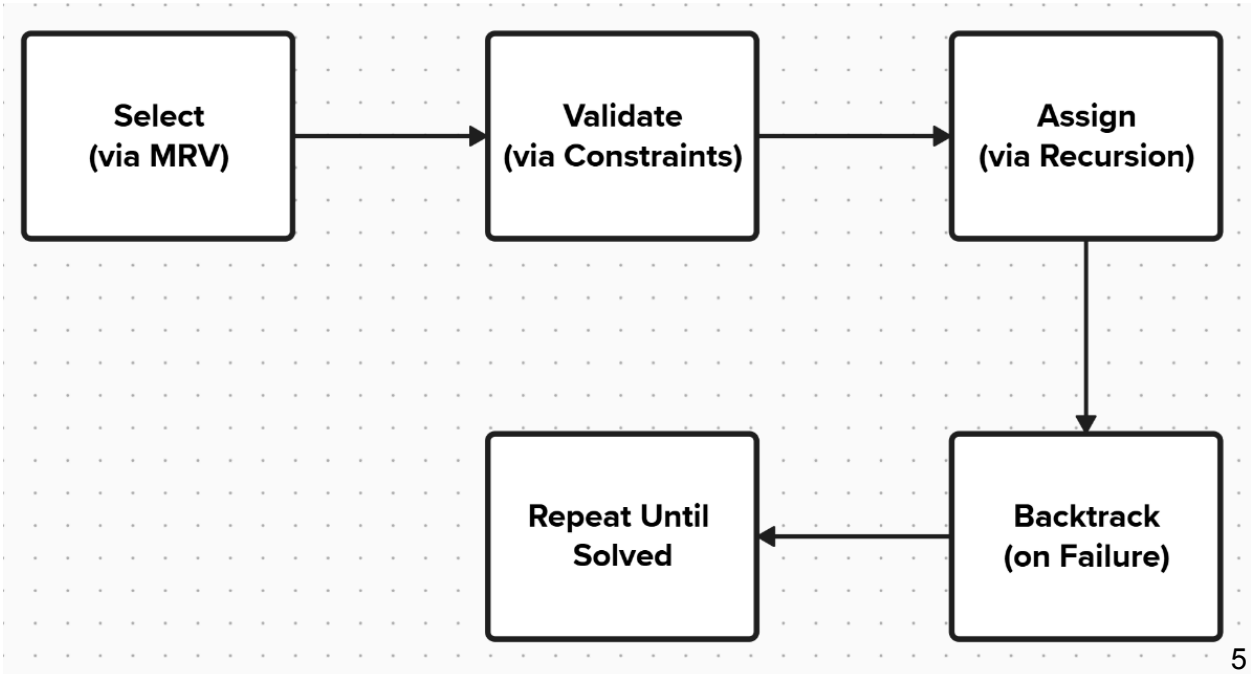
- **Recursive Dive:** The board state is updated, and the agent makes a recursive call to the main `solve_sudoku` function.
- **The Logic:** The program essentially asks itself: *"Assuming this specific number placement is correct, can we successfully solve the rest of the board?"* It then repeats Steps 1 and 2 for the next empty cell, diving deeper into the search tree.

Step 4: The Backtrack (Handling Dead Ends)

Because the agent is making educated guesses, it will inevitably traverse down a wrong path. A "dead end" occurs when the recursive function reaches a state where an empty cell has absolutely `0` valid options left.

- **Triggering the Reversal:** The recursive call returns `False`.
- **State Reset:** The algorithm performs the critical **backtrack** step by undoing its previous assignment (resetting the cell back to `0`).
- **Continuing the Search:** It then resumes its loop, trying the next valid number for that cell. If all 9 numbers have been exhausted and none work, it backtracks yet another step up the chain.

Summary of the Algorithmic Loop:



4) Importance of the Idea

We picked this project for a couple of reasons:

1. **Real-world CSP:** Sudoku is a great example of a Constraint Satisfaction Problem (CSP). Solving it is similar to solving real-world scheduling or planning problems where many rules have to be met at once.
2. **Seeing Heuristics In Action:** This project clearly shows how adding a smart trick like MRV can turn a slow, exhaustive search into a very fast and efficient one.

5) Discussion of Results

Our comparative testing between Standard Backtracking and MRV-optimized Backtracking revealed a fascinating reality about **Heuristic Overhead**:

- Surprisingly, Standard Backtracking was slightly faster. **Even with the Hard Grid**... This is because standard backtracking simply picks the very first empty cell it sees (minimal overhead). Since easy grids have few dead ends, it guesses correctly without much backtracking. The MRV algorithm, however, spends extra milliseconds scanning all 81 cells to calculate the options for every empty cell. Here, the overhead of "thinking" outweighed the benefits of the heuristic.
- To truly test MRV, we introduced a specially crafted grid **(The Evil Grid)** heavily constrained at the bottom but empty at the top. Standard Backtracking gets completely trapped, exploring millions of useless combinations in the top rows, taking a massive amount of time. MRV, however, immediately detects the heavy constraints, jumps to the most restricted cells, and solves the grid in a fraction of a second.

Conclusion:

The Evil Grid took **346.131 Seconds** to finish using the Standard Backtracking, **approximately 5.7 Minutes**, which is a crazy number ... However, while using MRV, it finished only in **4.42 seconds**, which means the **MRV optimized AI-Agent is 78 times Faster** than normal backtracking.

The MRV heuristic is absolutely essential for complex, highly-constrained problems, even if its computational overhead makes it slightly

Initial Problem (Evil Grid)

				3		8	5
		1		2			
			5		7		
		4				1	
	9						
5						7	3
		2		1			
				4			9

```
==================================================
 RACING ALGORITHMS: Standard vs Optimized MRV
==================================================

[AI Agent 1] Solving WITHOUT MRV (Standard Backtracking)...
 -> Finished in: 346.13100 seconds

[AI Agent 2] Solving WITH Optimized MRV Heuristic...
 -> Finished in: 4.41608 seconds
```

slower on trivial puzzles. To mitigate this overhead, we optimized our MRV function to instantly return any cell that has 0 or 1 options, significantly closing the speed gap on easier grids.

6) Results

Easy:

			2	6		7		1
6	8			7			9	
1	9				4	5		
8	2		1				4	
		4	6		2	9		
	5				3		2	8
		9	3				7	4
	4			5			3	6
7		3		1	8			

Final Solution (Easy Grid)

4	3	5	2	6	9	7	8	1
6	8	2	5	7	1	4	9	3
1	9	7	8	3	4	5	6	2
8	2	6	1	9	5	3	4	7
3	7	4	6	8	2	9	1	5
9	5	1	7	4	3	6	2	8
5	1	9	3	2	6	8	7	4
2	4	8	9	5	7	1	3	6
7	6	3	4	1	8	2	5	9

[Success] Puzzle solved in: 0.00067 seconds

Medium:

			6			4		
7					3	6		
				9	1		8	
	5		1	8				3
			3		6		4	5
	4		2					

Final Solution (Medium Grid)

1	8	9	6	2	5	4	3	7
7	2	5	8	4	3	6	9	1
4	3	6	7	9	1	5	8	2
3	6	7	4	5	2	8	1	9
2	5	4	1	8	9	7	6	3
9	1	8	3	7	6	2	4	5
8	4	3	2	1	7	9	5	6
5	7	1	9	6	4	3	2	8
6	9	2	5	3	8	1	7	4

[Success] Puzzle solved in: 0.00830 seconds

Hard:

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

Final Solution (Hard Grid)

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

[Success] Puzzle solved in: 1.01359 seconds

Unsolvable:

Initial Problem (Unsolvable Grid)

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	7

[AI Agent] Solving using Optimized MRV Backtracking...

[Error] No solution exists for the selected 'Unsolvable Grid'.

Original:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Final Solution (Original Grid)

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

[Success] Puzzle solved in: 0.00413 seconds

Evil:

					3		8	5
		1		2				
			5		7			
		4				1		
	9							
5							7	3
		2		1				
				4				9

Final Solution (Evil Grid)

9	8	7	6	5	4	3	2	1
2	4	6	1	7	3	9	8	5
3	5	1	9	2	8	7	4	6
1	2	8	5	3	7	6	9	4
6	3	4	8	9	2	1	5	7
7	9	5	4	6	1	8	3	2
5	1	9	2	8	6	4	7	3
4	7	2	3	1	9	5	6	8
8	6	3	7	4	5	2	1	9

[Success] Puzzle solved in: 4.48226 seconds

Try the AI Agent:

Free Topic

We chose **Local Beam Search** as our free topic.

7) Local Beam Search Explanation

For simplification, we like to think of it like this: instead of having just one person trying to find the highest point on a mountain in the fog (like the standard Hill-Climbing Algorithm), **you send out a team of k people.**
The cool part is that **this team talks to each other.** They all look at their possible next steps. Instead of everyone just taking their own best step, they **combine all their ideas into one big list.** Then, they look at the whole list and **pick the absolute best k moves overall.** Everyone then moves to those top spots. **If one person is stuck in a bad area, they just jump over** to where someone else has found a better path.
The algorithm only remembers these k states. **It keeps generating moves, picking the best k, and repeating until it finds the goal.**

- **Defining "Local":**

  The term "local" is used because it only evaluates **its current position and** its **immediate surroundings (its neighbors). It keeps absolutely no memory of the past path it took to get there.**

- **Defining "Beam":**

  Imagine a flashlight. Instead of trying to illuminate the entire search space at once (like Breadth-First Search, which consumes massive memory), **it restricts its search to a narrow "beam" of size k.** It **only shines light on the k most promising paths**, **leaving the rest in the dark to be discarded.**

8) Application Example (8-Queens Problem)

Below is a complete breakdown of how this algorithm approaches and solves the problem.

1. The Problem

The 8-Queens problem is a classic chess puzzle. You are given a standard 8x8

chessboard and 8 Queen pieces. The challenge is to place all 8 queens on the board so that **no two queens threaten each other**. According to chess rules, this means no two queens can share the same row, column, or diagonal line.

2. The Goal

To reach a "Complete State" (a fully populated board) where the total number of attacking pairs (conflicts) is exactly **zero**.

3. The Inputs

To run the Local Beam Search algorithm on this problem, we provide the following inputs:

- $N = 8$**:** The size of the board and the number of queens.
- K **(Beam Width):** The number of states the algorithm will keep in its memory at any given time (Ex, $k = 4$).
- **Max Iterations:** A safety limit (Ex, 1000 loops) to prevent the program from running infinitely if it gets completely stuck.

4. Execution Steps (From Start to Completion)

- **Step 1: Initialization (Starting the Beams)**

  The algorithm does not start with an empty board. Instead, **it generates k=4 completely random boards. Each of these 4 boards has 8 queens placed randomly** (with the restriction of one queen per column). At this stage, the boards are very messy, and many queens are attacking each other.

- **Step 2: Heuristic Evaluation (Scoring)**

  The algorithm **scans** each of the 4 boards and calculates its **"Heuristic Score" (h).** The score is simply **the number of attacking queen pairs**. For example, Board A might have 6 attacking pairs, Board B might have 4, etc.

- **Step 3: Goal Check**

  To save processing power. The algorithm looks at the current boards and asks: ***Does any board have a score of 0?*** **If yes, the problem is solved**, and the program halts immediately without doing unnecessary extra work. If no board is perfectly solved, it proceeds to the next step.

- **Step 4: Generate Successors (Exploring Moves)**

  Since the goal was not met, the algorithm must explore new possibilities. It does this by generating every possible "next move" for the current boards. A move is defined as sliding a single queen up or down to a different square in its own column. *Let us break down the mathematical scale of this step:*

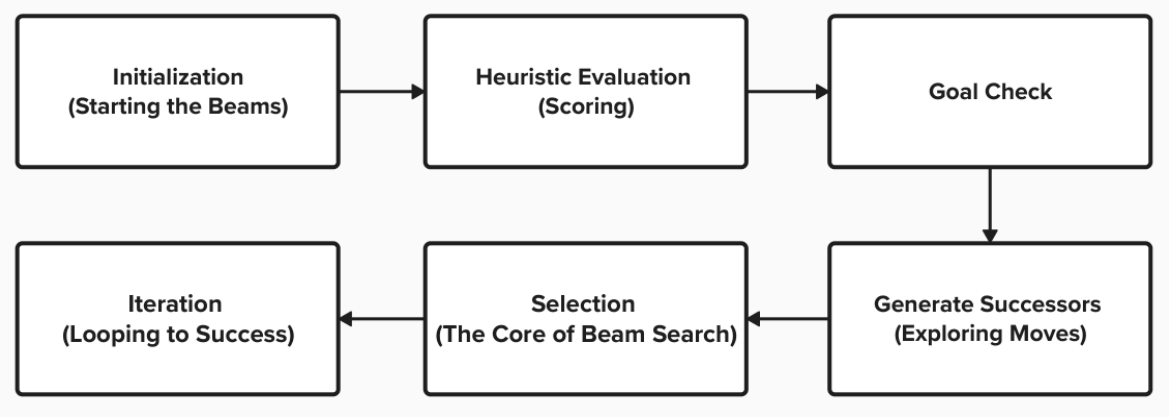
  - A single board has **8 Queens**.
  - Each queen can slide into **7 other empty squares** in her column.
  - Therefore, 1 single board generates $8 \times 7 = 56$ **new, slightly modified board states**.
  - Because the algorithm is tracking $k=4$ boards simultaneously, it generates a total massive pool of $56 \times 4 = 224$ **new board states** in this single step.
- **Step 5: Selection (The Core of Beam Search)**

  The algorithm **calculates the heuristic score (h) for all 224 new neighbor boards**. It then sorts them from best (lowest conflicts) to worst. Here is where the "Beam" happens: It strictly **selects the absolute best 4 boards** from the entire pool of 224 **and permanently deletes the rest from memory.**

- **Step 6: Iteration (Looping to Success)**

  The algorithm takes these newly selected top 4 boards and loops back to **Step2**. With every single loop, the queens are guided into safer, less crowded positions. Because it tracks 4 independent paths instead of just 1, **if one board gets permanently trapped in a bad layout (a Local Maximum),** the algorithm simply **discards it and relies on the other 3 boards** to keep making progress.

This cycle of **[*Score* → *Check Goal* → *Generate 224 Neighbors* → *Filter Top 4*]** repeats rapidly until one of the boards finally achieves exactly 0 conflicts, at which point the algorithm outputs the victorious board and terminates Others.



9) Evaluation of Local Beam Search

To fully evaluate Local Beam Search, we need to look at how it performs in terms of memory, speed, and reliability. Below is a structured evaluation highlighting its strengths and weaknesses.

Pros:

- **Excellent Space Complexity (Memory):**

  This is the biggest win for this algorithm. Regular search methods (like **Breadth-First Search**) can eat up all your computer's memory quickly because they save the whole search tree. Local Beam Search **strictly limits its memory** footprint to **only keep track of $\mathbf{k}$ states**.

- **Very Good Time Complexity (Speed):**

  At each step, it only generates the neighbors for the k states. This means it doesn't waste time exploring bad paths deeply. **It quickly jumps to the most promising options**, making it generally fast in finding a goal if one is nearby.

- **Dynamic Resource Allocation:**

  It dynamically **shares information among the $\mathbf{k}$ states**, allowing the algorithm to abandon unfruitful searches and focus entirely on the paths making the most progress.

Cons:

- **Not Complete:**

  Unlike standard Backtracking (which guarantees finding a solution eventually), Local Beam Search is not complete. **It might run forever or give up if all $\mathbf{k}$ states get stuck** in a bad spot and can't find a way out.

- **Not Optimal:**

  It doesn't care about finding the absolute shortest or "best" sequence of moves; **it just wants to reach a goal state.**

- **The Concentration Problem (Lack of Diversity):**

  A major flaw is that **all $\mathbf{k}$ states can end up crowding into the same small area of possibilities.** If that area happens to be a "local maximum" (a state with just a few stubborn errors but no real solution), they all get stuck together and stop making progress.

10) Summary

In this project, **we successfully built an intelligent Python agent to solve Sudoku puzzles by** modeling them as a **Constraint Satisfaction Problem (CSP).** Our primary approach utilized a recursive Backtracking Search enhanced **with an optimized Minimum Remaining Values (MRV) heuristic**.

Throughout the development phase, we conducted extensive comparative testing that yielded surprising insights. **We discovered the concept of "Heuristic Overhead," noting that on easy grids, standard backtracking was occasionally faster because the MRV calculations took extra time**. However, **by designing an "Evil Grid"** to specifically trap standard backtracking into a Search Space Explosion, **we proved the absolute necessity of MRV**, which solved the trap in milliseconds.

For our Free Topic, we explored the Local Beam Search algorithm. We initially attempted to use it to solve Sudoku, see appendix A. which led to a highly educational failure. The algorithm consistently became trapped in Local Maxima due to the rigid nature of Sudoku constraints. By pivoting our example to the 8-Queens problem, **we successfully demonstrated how Local Beam Search excels in problems with smoother search spaces,** ultimately proving that theoretical knowledge must be paired with practical experimentation.

11) Conclusion

Building this AI agent was a profound learning experience that went far beyond writing standard code. The most valuable lessons came from our errors, bottlenecks, and experimental failures.

In conclusion, this project demonstrated that choosing the right AI algorithm depends entirely on the mathematical nature of the state space. Backtracking combined with MRV is the scientifically superior choice for strict constraint puzzles like Sudoku, while Local Search algorithms are better reserved for continuous optimization and placement problems.

12) References

We relied heavily on our **course materials**, the primary textbook, and supplementary academic literature to formulate the problem and accurately detail the algorithms:

1. **Russell, S. J., & Norvig, P. (2010).** *Artificial Intelligence: A Modern Approach* (3rd ed.). Prentice Hall.
    - *Chapter 3:* Solving Problems by Searching (Concepts of States, Actions, and Goals).
    - *Chapter 4:* Beyond Classical Search (Local Beam Search definitions and the Stochastic variant, p. 125-126).
    - *Chapter 6:* Constraint Satisfaction Problems (CSPs, Backtracking, and the MRV heuristic).
2. **Simonis, H. (2005).** *Sudoku as a Constraint Problem*. In Proceedings of the 4th International Workshop on Modelling and Reformulating Constraint Satisfaction Problems (CP 2005). (Used as a supplementary reference for framing Sudoku effectively as a CSP).
3. **Python Software Foundation.** *Python Language Reference, version 3.x*. Available at http://www.python.org (Referenced for time and copy modules).

Try Speed Comparison:

Try the AI Sudoku Solver agent:

Soft-copy of The Data:

13) Appendix A

Experimental Application of Local Beam Search on Sudoku

A.1 Overview of the Experiment

While researching our Free Topic, we asked a simple question: *Can Local Beam Search (LBS) solve our Sudoku puzzle?* We wrote a script to test this out. The experiment failed to solve the Sudoku, but it taught us a very valuable lesson about why certain AI algorithms only work for specific types of problems.

A.2 How We Tested It

Instead of building the Sudoku board cell by cell (like Backtracking does), we tried a guessing approach:

1. Random Start: We took the unsolved Sudoku board and filled all the empty squares with completely random numbers.
2. Scoring: We counted the "errors." An error was any duplicate number in a row, column, or 3x3 box. The goal was to reach 0 errors.
3. Making Moves: The AI randomly picked cells and changed their numbers, hoping to lower the error score.
4. Keeping the Best: Because it's a "Beam" search (with $k=4$), the AI looked at all the new board combinations, kept the 4 best boards (the ones with the fewest errors), and threw the rest away. It then repeated this process.

A.3 The Result (Getting Stuck)

When we ran the program, the same thing happened every time:

- **A Great Start:** At first, the algorithm was very fast. It quickly dropped the errors from 50+ down to just single digits.
- **Hitting a Wall:** Then, it completely froze. It would reach a board with only 2 or 3 errors left, but it could never reach 0. Even after thousands of loops, it was permanently stuck on the same flawed boards.

A.4 Why It Failed (What We Learned)

By analyzing this failure, we learned two major AI concepts:

1. **The "Local Maximum" Trap:** Imagine climbing a mountain in the fog and reaching the top of a small hill. Every step you take from there goes downward, so you stop, thinking you are at the highest peak.
    - **In our Sudoku:** The AI reached a board with only 2 errors. To fix those final 2 errors, it would have to temporarily mess up other numbers, increasing the errors to 5 or 6. Because the LBS algorithm is designed to *only* accept moves that lower the score, it refused to make things worse before making them better. It got trapped on the "small hill."
2. **Sudoku's Strict Rules vs. Guessing:** LBS is a great algorithm for problems where small changes lead to steady, smooth improvements (like sliding pieces in the 8-Queens problem). But Sudoku's rules are too rigid. Changing a single number to fix a row will almost instantly break a column or a box. You can't just "guess and fix" Sudoku.

Conclusion:

This failed experiment practically proved our main project choice. It showed that for strict logic puzzles like Sudoku, you need an organized, systematic algorithm that can easily step back and try a different path—which is exactly why **Backtracking with MRV** is the perfect solution.

14) Appendix. B

The Python Code for the AI Sudoku Solver Agent:

```
import time

import copy # Import the copy module for deep copying lists


def is_valid(grid, row, col, num):

    """ Check if placing a number breaks Sudoku rules (Row, Col, 3x3 Box)
"""

    for i in range(9):

        if grid[row][i] == num:

            return False

    for i in range(9):

        if grid[i][col] == num:

            return False

    start_row, start_col = 3 * (row // 3), 3 * (col // 3)

    for i in range(3):

        for j in range(3):

            if grid[start_row + i][start_col + j] == num:

                return False

    return True


def get_mrv_cell(grid):
```

```
    """ Optimized MRV: Returns immediately if a cell has 0 or 1 options.
"""
    min_options = 10
    best_cell = None
    for r in range(9):
        for c in range(9):
            if grid[r][c] == 0:
                        options_count = sum(1 for num in range(1, 10) if
is_valid(grid, r, c, num))


                # Hit a dead end? Return immediately to force backtrack!
                if options_count == 0:
                    return (r, c)


                 # Found a cell with exactly 1 option? It's the best
possible choice.
                if options_count == 1:
                    return (r, c)


                if options_count < min_options:
                    min_options = options_count
                    best_cell = (r, c)
    return best_cell
```

```
def solve_sudoku(grid):
        """ Core Backtracking Search algorithm utilizing optimized MRV
heuristic """
    cell = get_mrv_cell(grid)
    if not cell:
        return True # Puzzle Solved


    row, col = cell
    for num in range(1, 10):
        if is_valid(grid, row, col, num):
            grid[row][col] = num
            if solve_sudoku(grid):
                return True
            grid[row][col] = 0 # Backtrack
    return False


def print_styled_grid(grid, title="Sudoku Grid"):
    """ A mobile/Colab friendly compact grid display """
    print(f"\n--- {title} ---")
    for r in range(9):
        if r % 3 == 0:
```

```
            print("-" * 25)


        row_str = "| "
        for c in range(9):
            val = str(grid[r][c]) if grid[r][c] != 0 else "."
            row_str += val + " "
            if (c + 1) % 3 == 0:
                row_str += "| "
        print(row_str.strip())
    print("-" * 25)


# ========================================
# Grids Definition
# ========================================
easy_grid = [
    [0, 0, 0, 2, 6, 0, 7, 0, 1],
    [6, 8, 0, 0, 7, 0, 0, 9, 0],
    [1, 9, 0, 0, 0, 4, 5, 0, 0],
    [8, 2, 0, 1, 0, 0, 0, 4, 0],
    [0, 0, 4, 6, 0, 2, 9, 0, 0],
    [0, 5, 0, 0, 0, 3, 0, 2, 8],
    [0, 0, 9, 3, 0, 0, 0, 7, 4],
```

```
    [0, 4, 0, 0, 5, 0, 0, 3, 6],
    [7, 0, 3, 0, 1, 8, 0, 0, 0]
]

medium_grid = [
    [0, 0, 0, 6, 0, 0, 4, 0, 0],
    [7, 0, 0, 0, 0, 3, 6, 0, 0],
    [0, 0, 0, 0, 9, 1, 0, 8, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 5, 0, 1, 8, 0, 0, 0, 3],
    [0, 0, 0, 3, 0, 6, 0, 4, 5],
    [0, 4, 0, 2, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0]
]

hard_grid = [
    [8, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 3, 6, 0, 0, 0, 0, 0],
    [0, 7, 0, 0, 9, 0, 2, 0, 0],
    [0, 5, 0, 0, 0, 7, 0, 0, 0],
    [0, 0, 0, 0, 4, 5, 7, 0, 0],
```

```
    [0, 0, 0, 1, 0, 0, 0, 3, 0],
    [0, 0, 1, 0, 0, 0, 0, 6, 8],
    [0, 0, 8, 5, 0, 0, 0, 1, 0],
    [0, 9, 0, 0, 0, 0, 4, 0, 0]
]

unsolvable_grid = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 7] # This 7 conflicts with the 7 in col 8
]

evil_grid = [
    [0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 3, 0, 8, 5],
    [0, 0, 1, 0, 2, 0, 0, 0, 0],
```

```
    [0, 0, 0, 5, 0, 7, 0, 0, 0],
    [0, 0, 4, 0, 0, 0, 1, 0, 0],
    [0, 9, 0, 0, 0, 0, 0, 0, 0],
    [5, 0, 0, 0, 0, 0, 0, 7, 3],
    [0, 0, 2, 0, 1, 0, 0, 0, 0],
    [0, 0, 0, 0, 4, 0, 0, 0, 9]
]


# ========================================
# Main Execution Menu
# ========================================
if __name__ == "__main__":
    original_initial_grid = [
        [5, 3, 0, 0, 7, 0, 0, 0, 0],
        [6, 0, 0, 1, 9, 5, 0, 0, 0],
        [0, 9, 8, 0, 0, 0, 0, 6, 0],
        [8, 0, 0, 0, 6, 0, 0, 0, 3],
        [4, 0, 0, 8, 0, 3, 0, 0, 1],
        [7, 0, 0, 0, 2, 0, 0, 0, 6],
        [0, 6, 0, 0, 0, 0, 2, 8, 0],
        [0, 0, 0, 4, 1, 9, 0, 0, 5],
        [0, 0, 0, 0, 8, 0, 0, 7, 9]
```

```
    ]


    print("\n--- Sudoku Solver Menu ---")
    print("1. Easy Grid")
    print("2. Medium Grid")
    print("3. Hard Grid")
    print("4. Unsolvable Grid")
    print("5. Original Grid")
    print("6. Evil Grid")


    while True:
        try:
            choice = int(input("Enter your choice (1-6): "))
            if 1 <= choice <= 6:
                break
            else:
                print("Invalid choice. Please enter a number between 1 and
6.")
        except ValueError:
            print("Invalid input. Please enter a number.")


    selected_grid = []
```

```
    grid_name = ""
    if choice == 1:
        selected_grid = copy.deepcopy(easy_grid)
        grid_name = "Easy Grid"
    elif choice == 2:
        selected_grid = copy.deepcopy(medium_grid)
        grid_name = "Medium Grid"
    elif choice == 3:
        selected_grid = copy.deepcopy(hard_grid)
        grid_name = "Hard Grid"
    elif choice == 4:
        selected_grid = copy.deepcopy(unsolvable_grid)
        grid_name = "Unsolvable Grid"
    elif choice == 5:
        selected_grid = copy.deepcopy(original_initial_grid)
        grid_name = "Original Grid"
    elif choice == 6:
        selected_grid = copy.deepcopy(evil_grid)
        grid_name = "Evil Grid"

            print_styled_grid(selected_grid,   title=f"Initial   Problem
({grid_name})")
```

```
    print("\n[AI Agent] Solving using Optimized MRV Backtracking...")
    start_time = time.time()

    if solve_sudoku(selected_grid):
                print_styled_grid(selected_grid, title=f"Final Solution ({grid_name})")
                print(f"\n[Success] Puzzle solved in: {time.time() - start_time:.5f} seconds")
    else:
               print(f"\n[Error] No solution exists for the selected '{grid_name}'.")
```

14) Task Assignment Page

#	Student Name	Task Assigned	Percentage Contribution	Signature
1	Ramy Said Eliwa Elsayed Mohammed	**Sudoku:** Problem Formulation & Tools Setup. **Free Topic:** LBS Evaluation, Appendix A & Conclusion.	33.3%	.....
2	David George Helmy Ekladious Shenoda	**Sudoku:** Results Analysis & Performance Comparison. **Free Topic:** LBS Execution Steps & Successor Generation.	33.3%	.....
3	Ahmed Mohammed Abdelrazik	**Sudoku:** Backtracking Logic & MRV Heuristic Implementation. **Free Topic:** Local Beam Search Intro & 8-Queens Formulation.	33.3%	.....